## CX651 — Assignment 9

## Java Execution Traces: Predict, Run, Explain

### Overview

In this assignment, you will work through five short, pre-written Java programs. For each program, predict its output before running it, compile and run it, and explain your observations using the relevant Java rule. Run each program as provided, except for the modification explicitly requested in Q8. A clear explanation of the rule is sufficient; formal citations are not required.

### General Instructions

Download this document and the five Java files to your computer. Type your answers in this document and submit the completed file to Gradescope. Use a JDK that provides javac and java, and run commands from the directory containing the Java files. The following files are provided:

- `Trace1.java` — autoboxing and the == operator
- `Trace2.java` — pass-by-value with a mutable object
- `Trace3.java` — checked vs. unchecked exceptions
- `Trace4.java` — polymorphism and virtual method dispatch
- `Trace5.java` — generics and type erasure

### How to Run

Compile and run each file individually from the command line:

```
javac Trace1.java && java Trace1
javac Trace2.java && java Trace2
# ... and so on for Trace3, Trace4, Trace5
```

For each trace, complete the three-column table in order: prediction, actual output, and a brief explanation. Predict before running. Answer every numbered question in its own Answer area as well; when a question requests output, repeat that output there even if it already appears in the table. Keep the question labels and their order unchanged.

**Q1.** Run javac -version in your terminal and paste its exact output in the Answer area directly below. This identifies the Java compiler version used for your work.

```
javac -version
```

Your Answer:

javac 21.0.12

### Trace 1 — Autoboxing and the == Operator

This program uses autoboxing to create Integer references and compares reference equality with numeric equality.

```
public class Trace1 {
    public static void main(String[] args) {
        Integer a = 100;
        Integer b = 100;
        Integer c = 200;
        Integer d = 200;
        System.out.println(a == b);
        System.out.println(c == d);
        System.out.println(a.equals(b));
        System.out.println(c.equals(d));
    }
}
```

*Complete the table: predict the output before running, then run it, then explain the result in terms of the Java rule involved.*

| Predicted result | Actual result | Your explanation |
|---|---|---|
| 1)true<br>2)false<br>3)true<br>4)true | 1)true<br>2)false<br>3)true<br>4)true | compares object references for Integer objects. Autoboxing the constant 100 uses a cached Integer, so a and b refer to the same object. Caching is guaranteed for this range;<br>For small integers from -128 through 127, Java guarantees that boxing reuses the same cached Integer object. So, a and b point to the same object, and a == b is true.<br>for 200, it isn't, 200 is outside that guaranteed range. Java created separate Integer objects for c and d, so c == d is false. Java implementations may cache additional values, so you shouldn't rely on == giving false for 200 in every environment.<br>The values are still equal, which is why c.equals(d) is true. Use equals () to compare Integer values; use == only when you mean to compare object identity. |

**Q2.** Begin your answer with the four boolean values printed by Trace1, in order, one per line. Then explain your observed results for a == b and c == d in terms of Integer caching and reference identity.

*Answer:*
true
false
true
true

Autoboxing 100 is within Java's guaranteed Integer cache range of **-128 to 127**, so both variables refer to the same cached object and a == b is true. 200 is outside that guaranteed range; in this run, c and d refer to **distinct** objects, so c == d is false. == compares references, not numeric values.

**Q3.** What method should you use instead of == to reliably compare two non-null Integer values for numeric equality, regardless of their value? Name the method using empty parentheses, and explain why it works. You may also include an example call with an argument.

*Answer:*
Use **.equals()** to compare two non-null Integer values. It compares the numeric values they contain, rather than whether the references point to the same object. For example, c.equals(d) returns true when both hold 200.

### Trace 2 — Pass-by-Value with a Mutable Object

This program passes a primitive value and an object reference to methods that modify a parameter, mutate an object, or reassign a reference.

```
public class Trace2 {
    static void tryToDouble(int x) {
        x = x * 2;
    }
    static void appendWorld(StringBuilder sb) {
        sb.append(" world");
    }
    static void replaceRef(StringBuilder sb) {
        sb = new StringBuilder("replaced");
    }
    public static void main(String[] args) {
        int n = 5;
        tryToDouble(n);
        System.out.println(n);
        StringBuilder greeting = new StringBuilder("hello");
        appendWorld(greeting);
        System.out.println(greeting);
        replaceRef(greeting);
        System.out.println(greeting);
    }
}
```

*Complete the table: predict the output before running, then run it, then explain the result in terms of the Java rule involved.*

| Predicted result | Actual result | Your explanation |
|---|---|---|
| 1)5<br>2) hello world<br>3) hello world | 1)5<br>2) hello world<br>3) hello world | Java passes arguments by value. tryToDouble receives a copy of the primitive n, so doubling its local copy doesn't change the original n, which stays 5.<br><br>For the StringBuilder calls, the method receives a copy of the reference. appendWorld uses that copied reference to modify the same object,<br>so greeting becomes hello world. replaceRef only makes its local parameter refer to a new builder; it doesn't change the caller's greeting reference, so it remains hello world. |

**Q4.** Java is described as "pass-by-value." Run the program and examine the three lines it prints. To make your grader's job easier, begin your answer by stating the three exact output values you observed, in order (one value per bullet or line). Then explain what that phrase means for a primitive (`int` n) vs. for an object reference (StringBuilder greeting). Why did appendWorld() change the content but replaceRef() did not?

*Answer:*

Java is pass by value: a method receives a copy of the value passed to it.
For int n, the value copied is the integer itself. tryToDouble changes only its local copy, so n remains 5.
For StringBuilder greeting, the value copied is a reference to the builder. appendWorld () uses that copied reference to mutate the same builder object, so its content changes to hello world. replaceRef () instead assigns its local parameter to a new builder; that does not change the
caller's greeting reference, so the original builder still contains hello world

**Q5.** Omitted. No response is required. Question numbering is preserved for consistency with the grading setup.

### Trace 3 — Checked vs. Unchecked Exceptions

This program catches one exception but lets another propagate — study how the compiler enforces exception handling.

```
import java.io.*;
public class Trace3 {
    static void riskyIO() throws IOException {
        throw new IOException("disk full");
    }
    static void riskyRuntime() {
        throw new IllegalArgumentException("bad input");
    }
    public static void main(String[] args) {
        try {
            riskyIO();
        } catch (IOException e) {
            System.out.println("Caught: " + e.getMessage());
        }
        riskyRuntime();
        System.out.println("This line runs only if no exception escapes.");
    }
}
```

*Complete the table: predict the output before running, then run it, then explain the result in terms of the Java rule involved.*

| Predicted result | Actual result | Your explanation |
|---|---|---|
| 1)Caught: disk full<br><br>2)uncaught IllegalArgumentException: bad input stack trace. | 1)Caught: disk full<br><br>2)Uncaught IllegalArgumentException: bad input stack trace. | IOException is a checked exception, so riskyIO () declares it with throws, and main catches it. The catch prints Caught: disk full, and execution continues.<br><br>IllegalArgumentException is unchecked, so it does not need to be declared or caught. Here it escapes main, ends the program, and prevents This line runs only if no exception escapes. from printing. |

**Q6.** Describe the two output events when Trace3 runs: the message printed by the catch block and the uncaught-exception diagnostic. State the number of events, treating the entire stack trace as one diagnostic block rather than counting its individual lines. Identify the final exception and explain whether the last System.out.println statement executes.

*Answer:*

1. The catch block prints: Caught: disk full
2. An uncaught-exception diagnostic block reports java.lang.IllegalArgumentException: bad input and shows the stack trace.

The final exception is the uncaught IllegalArgumentException. The last System.out.println does **not** execute because the exception escapes main and terminates the program.

**Q7.** riskyIO() declares 'throws IOException' and the caller wraps the call in try/catch. riskyRuntime() declares nothing and its caller uses no try/catch. Explain the Java rule that requires handling for one but not the other. Where in the class hierarchy does that distinction lie?

*Answer:*

The **compiler** enforces the checked-exception rule: because IOException is a checked exception, code that calls riskyIO() must catch it or declare it with throws. IllegalArgumentException extends RuntimeException, making it unchecked; the compiler doesn't require callers to catch or declare it.

**Q8.** For this question only, make a temporary copy of Trace3.java in a separate directory, keeping the filename Trace3.java. In that copy, replace the entire try/catch statement in main with riskyIO();. Keep the method declarations unchanged and do not add a throws clause to main. This modification is permitted despite the file header comment. Compile the modified file with javac and paste the exact compiler error. Explain why the call to riskyRuntime() does not require the same catch-or-declare treatment. Use the original file for the other questions.

*Answer:*

```
Trace3.java:18: error: unreported exception IOException; must be caught or declared to be thrown
      riskyIO();
          ^
1 error
```

IOException is a checked exception, so Java requires the call to be handled with try/catch or for the enclosing method to declare it with throws. Since main does neither in the modified copy, compilation fails.
riskyRuntime() throws IllegalArgumentException, which is unchecked because it extends RuntimeException. Unchecked exceptions don't require callers to catch or declare them. The original Trace3.java was not modified; the temporary copy was removed after compilation.

### Trace 4 — Polymorphism and Virtual Method Dispatch

This program builds an inheritance hierarchy and calls an inherited method on objects of different runtime classes through Animal references.

```
public class Trace4 {
    static class Animal {
        String sound() { return "..."; }
        void describe() {
            System.out.println("I am " + getClass().getSimpleName()
                + " and I say " + sound());
        }
    }
    static class Dog extends Animal {
        @Override String sound() { return "woof"; }
    }
    static class Cat extends Animal {
        @Override String sound() { return "meow"; }
    }
    static class Kitten extends Cat {
        @Override String sound() { return "mew"; }
    }
    public static void main(String[] args) {
        Animal[] zoo = { new Animal(), new Dog(), new Cat(), new Kitten() };
        for (Animal a : zoo) {
            a.describe();
        }
    }
}
```

*Complete the table: predict the output before running, then run it, then explain the result in terms of the Java rule involved.*

| Predicted result | Actual result | Your explanation |
|---|---|---|
| 1)I am Animal and I say ...<br>2)I am Dog and I say woof<br>3)I am Cat and I say meow<br>4) I am Kitten and I say mew | 1)I am Animal and I say ...<br>2)I am Dog and I say woof<br>3)I am Cat and I say meow<br>4) I am Kitten and I say mew | Java uses dynamic method dispatch for overridden instance methods. Although the array and loop variable are typed as Animal, the call to sound () uses the runtime class of each object. Thus Dog, Cat, and Kitten each use their own override (Kitten's override takes precedence over Cat's). getClass().getSimpleName() likewise reports each object's actual class. |

**Q9.** Write the four lines of output this program produces.

*Answer:*

I am Animal and I say ...
I am Dog and I say woof
I am Cat and I say meow
I am Kitten and I say mew

**Q10.** describe() is defined only in Animal, yet it prints different sound() values for each object. Explain how the JVM determines which sound() method to call at runtime. What bytecode instruction handles this dispatch, and what information does the JVM use to select the method? To inspect the bytecode after compiling, run: javap -c 'Trace4$Animal'. The single quotes protect the dollar sign in Bash, zsh, and PowerShell; in Windows Command Prompt, use double quotes instead.

*Answer:*
Bytecode for describe() contains

11: invokevirtual #25  // Method sound:()Ljava/lang/String;

At runtime, the invokevirtual instruction uses the receiver's actual class and **selects** the most-specific override of the method. A JVM commonly implements this virtual dispatch using a vtable.

**Q11.** getClass().getSimpleName() returns different values for each object even though the call appears inside a method defined in Animal. Explain why it reports the runtime class of the receiver rather than the declared reference type. Distinguish this behavior from overriding sound().

*Answer:*

getClass() is an instance method inherited from Object. When called on this, it returns a Class object describing the receiver's actual runtime class—not the declared type of a variable that may refer to it. In describe(), this can therefore be an Animal, Dog, Cat, or Kitten, and getSimpleName() returns that class's simple name.
This differs from sound(): getClass() itself is not overridden here; the same inherited method reports the object's runtime class. sound() is overridden in the subclasses, so runtime dispatch selects a different implementation based on that class.

### Trace 5 — Generics and Type Erasure

This program uses generic types and then inspects them at runtime using reflection.

```
import java.util.*;
public class Trace5 {
    static <T> void printFirstType(List<T> list) {
        if (!list.isEmpty()) {
            Object first = list.get(0);
            System.out.println(first.getClass().getName());
        }
    }
    public static void main(String[] args) {
        List<String>  strings  = new ArrayList<>();
        List<Integer> integers = new ArrayList<>();
        List<Double>  doubles  = new ArrayList<>();
        strings.add("hello");
        integers.add(42);
        doubles.add(3.14);
        printFirstType(strings);
        printFirstType(integers);
        printFirstType(doubles);
        System.out.println(strings.getClass() == integers.getClass());
        System.out.println(strings.getClass().getName());
    }
}
```

*Complete the table: predict the output before running, then run it, then explain the result in terms of the Java rule involved.*

| Predicted result | Actual result | Your explanation |
|---|---|---|
| java.lang.String<br>java.lang.Integer<br>java.lang.Double<br>true<br>java.util.ArrayList | java.lang.String<br>java.lang.Integer<br>java.lang.Double<br>true<br>java.util.ArrayList | first.getClass() reports the runtime class of each stored object, so it prints String, Integer, and Double. But Java generics use type erasure: the type arguments String and Integer are not distinct runtime classes for these lists.<br>Both strings and integers are ArrayList objects, so their getClass() values are equal, and the class name is java.util.ArrayList. |

**Q12.** Write the five lines of output this program produces.

*Answer:*

java.lang.String
java.lang.Integer
java.lang.Double
true
java.util.ArrayList

**Q13.** The last two lines compare the classes of a List<`String`> and a List<Integer> and print the class name of strings. What do these two lines reveal about how generic types are represented at runtime? Define the term 'type erasure' in your own words.

*Answer:*

At **compile time**, Java checks the generic types for correctness, then **type erasure** removes the type arguments (such as <String> and <Integer>) from the compiled code. At runtime,
both List<String> and List<Integer> are just ArrayList; their type arguments are erased,
so getClass() reports the same class for both.

**Q14.** Given that generic type information is erased at runtime, how does the Java compiler still enforce type safety for code like 'strings.add(42)'? At what point is the type check performed?

*Answer:*

The compiler enforces generic type safety **at compile time**, before type erasure. For example, it **rejects** strings.add(42) because strings is a List<String>, and 42 is not a String. When reading an item, the compiler may also insert a **cast** from the erased type (Object) back to String, so your code can use the result as a String.

## Submission

Submit one completed document named A09_<BU-ID>.docx or A09_<BU-ID>.pdf containing:

1. For each of the five traces: the completed three-column table (predicted output, actual output, explanation) followed by your written answers to each question.
2. Your answer to Q1, including the exact javac -version output.

Do not submit separate .java or .class files. You may leave the supplied source-code listings in this document; you do not need to copy them into your answers.

Submit via the Gradescope assignment link posted on the course site.

Type your answers in the provided Answer areas. Submit the DOCX directly, or export a PDF with selectable text using Word's Save As or Export options. Submit only one format. Do not submit screenshots or a scanned document.